

Trace visualization with StarVZ

Lucas Mello Schnorr, UFRGS

Journée thématique Analyse de performance GDR C4P
Inria Saclay, bâtiment Turing, salle Gilles Kahn
Palaiseau, France, June 22nd, 2026



Propaganda

Who am I?

Lucas Mello Schnorr

Prof. INF/UFRGS & PPGC

<http://www.inf.ufrgs.br/~schnorr>

schnorr@inf.ufrgs.br



Academic Career Computer Science

- Undergraduate UFSM (2003), Master (2005)
- Co-advised Phd between UFRGS and INPG, Grenoble, France (2009)
- CNRS Postdoc (2010 – 2012)
- Inria Postdoc (2016/2017)

General overview of the **Porto Alegre** city



Instituto de Informática em Porto Alegre (RS)

General overview of the **Porto Alegre** city



Historic Downtown **Porto Alegre** city



Instituto de Informática em Porto Alegre (RS)



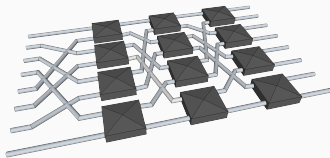
Informatics Institute gardens

GPPD - Parallel and Distributed Processing Group

URL:

<http://www.inf.ufrgs.br/gppd/site/>

Logo:



Main research axis

- High Performance Computing
- Computer Architecture
- Performance Analysis
- Cloud Computing

Computational Assets (PCAD)

URL: <http://pcad.inf.ufrgs.br/>

Around ≈ 50 nodes, $\approx 1000+$ CPU cores and $\approx 100000+$ GPU cores



Informatics Institute's *Data Center*, UFRGS

Work Group

- Education for HPC DevOps
- SLURM, NFS, LDAP, ...

Introduction

Performance Analysis in HPC: typical objectives

Main objectives of performance analysis

Performance Analysis in HPC: typical objectives

Main objectives of performance analysis

Understand application behavior

- Identify performance **bottlenecks** (CPU, memory, network, I/O)
- Understand **comm.** patterns between processes and threads
- Evaluate the use of available **computational resources**

Performance Analysis in HPC: typical objectives

Main objectives of performance analysis

Understand application behavior

- Identify performance **bottlenecks** (CPU, memory, network, I/O)
- Understand **comm.** patterns between processes and threads
- Evaluate the use of available **computational resources**

Guide optimization

- Focus efforts on parts with the **greatest impact** (*hotspots*)
- Validate the effect of **code modifications**
- Support **scheduling** and execution configuration decisions

Performance Analysis in HPC: typical objectives

Main objectives of performance analysis

Understand application behavior

- Identify performance **bottlenecks** (CPU, memory, network, I/O)
- Understand **comm.** patterns between processes and threads
- Evaluate the use of available **computational resources**

Guide optimization

- Focus efforts on parts with the **greatest impact** (*hotspots*)
- Validate the effect of **code modifications**
- Support **scheduling** and execution configuration decisions

Ensure quality and reproducibility

- Produce **reproducible** and verifiable analyses
- Compare performance across different **platforms**
- Generate evidence for **scientific publications**

Performance Analysis in HPC: the problem

Performance analysis is **essential** for efficient resource usage

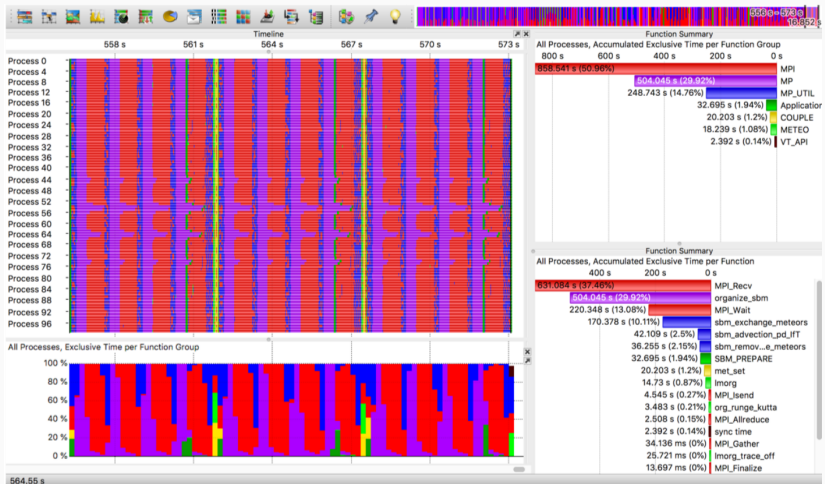
Performance Analysis in HPC: the problem

Performance analysis is **essential** for efficient resource usage

Traditional approach: **monolithic** tools

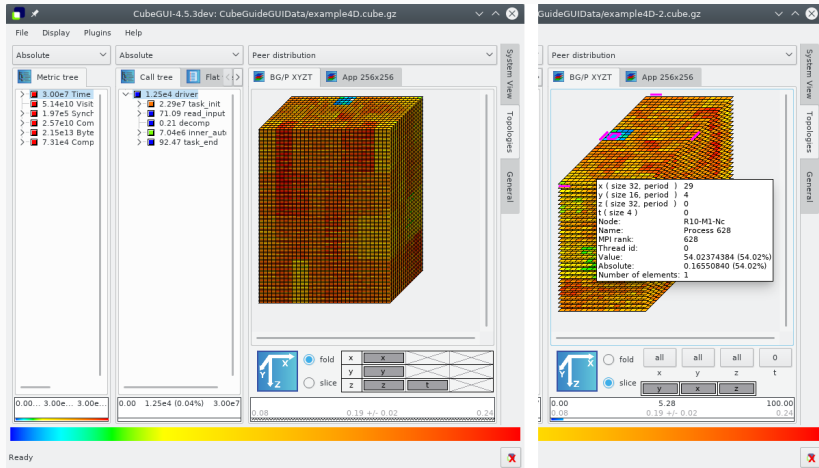
- Limited analytical **expressiveness**
- Low **extensibility** and adaptability
- Difficulty integrating with modern analysis workflows

Some examples: Vampir 1/4



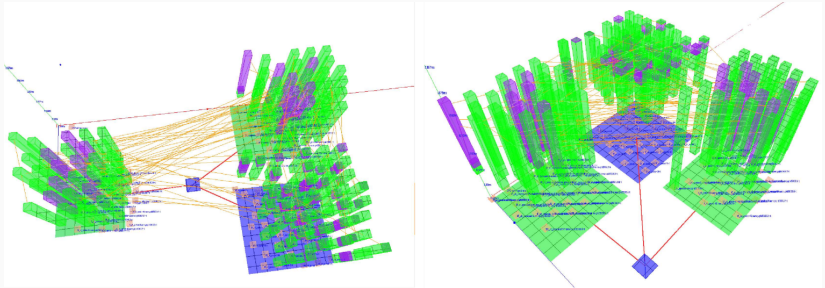
Williams, William, and Holger Brunst. "Parallel performance engineering using Score-P and Vampir." Companion of the 2023 ACM/SPEC International Conference on Performance Engineering. 2023.

Some examples: Scalasca Cube 2/4



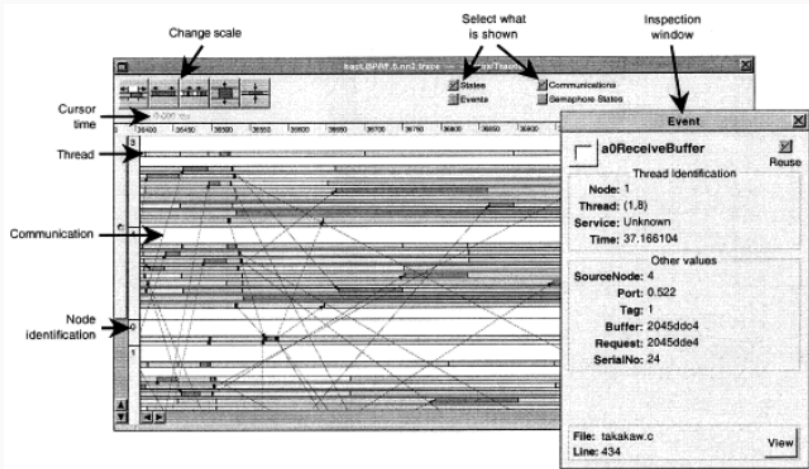
Zhukov, Ilya, et al. "Scalasca v2: Back to the future." Tools for High Performance Computing 2014: Proceedings of the 8th International Workshop on Parallel Tools for High Performance Computing, October 2014, HLRS, Stuttgart, Germany. Cham: Springer International Publishing, 2015.

Some examples: Triva 3/4



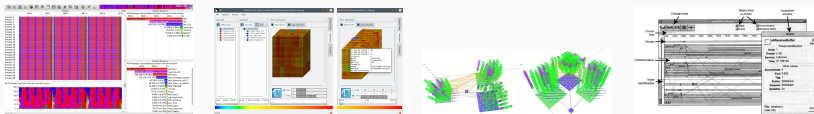
Schnorr, Lucas Mello, Guillaume Huard, and Philippe OA Navaux. "Triva: Interactive 3d visualization for performance analysis of parallel applications." *Future Generation Computer Systems* 26.3 (2010): 348-358.

Some examples: Page 4/4



Flexible performance visualization of parallel and distributed applications. J. Chassin de Kergommeaux, B. de Oliveira Stein Future Generation Computer Systems 19 (2003) 735-747.

Performance Analysis in HPC: the consequences



- Limited analysis of **execution traces** and performance profiles
- Slow optimization cycle
- Difficulty performing automations
 - For example via LLMs or SLMs
 - Or even via traditional code analysis methods
- Poorly reproducible
 - We don't have the “source code” of each specific view

Data Science + [possibly?] LLMs as a modern alternative

Emergence of data science and big data tools

Data Science + [possibly?] LLMs as a modern alternative

Emergence of **data science** and **big data** tools

New possibilities

- Expressive and flexible analysis of **execution traces**
- Support for a huge variety of application types
 - Traditional Bulk-Synchronous Parallel (BSP, MPI, MPI+X)
 - **Task-based** (StarPU, Parsec, OpenMP tasks)
- Handling **scalability** challenges
 - Huge volume of traced data

Data Science + [possibly?] LLMs as a modern alternative

Emergence of **data science** and **big data** tools

New possibilities

- Expressive and flexible analysis of **execution traces**
- Support for a huge variety of application types
 - Traditional Bulk-Synchronous Parallel (BSP, MPI, MPI+X)
 - **Task-based** (StarPU, Parsec, OpenMP tasks)
- Handling **scalability** challenges
 - Huge volume of traced data

Beyond performance analysis

- **Reproducibility** of experiments and analyses
- **Scientific transparency** in results

1. Present the StarVZ R package

- R-Based Visualization Techniques for Task-Based Applications
- <https://cran.r-project.org/web/packages/starvz/>

2. Some recent usage examples

StarVZ

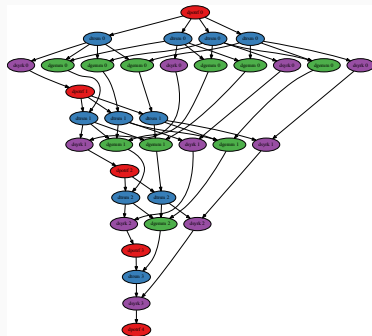
Task Parallelism in StarPU

Applications are represented as task graphs

- Application code is sequential (*Sequential Task Flow* – STF)

Example: Block Cholesky factorization

```
for (k = 0; k < N; k++) {  
    DPOTRF(RW,A[k][k]);  
    for (i = k+1; i < N; i++)  
        DTRSM(RW,A[i][k], R,A[k][k]);  
    for (i = k+1; i < N; i++) {  
        DSYRK(RW,A[i][i], R,A[i][k]);  
        for (j = k+1; j < i; j++)  
            DGEMM(RW,A[i][j],  
                R,A[i][k], R,A[j][k]);  
    }  
}
```



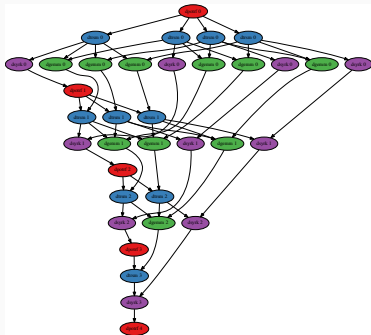
Task Parallelism in StarPU

Applications are represented as task graphs

- Application code is sequential (*Sequential Task Flow* – STF)

Example: Block Cholesky factorization

```
for (k = 0; k < N; k++) {  
    DPOTRF(RW,A[k][k]);  
    for (i = k+1; i < N; i++)  
        DTRSM(RW,A[i][k], R,A[k][k]);  
    for (i = k+1; i < N; i++) {  
        DSYRK(RW,A[i][i], R,A[i][k]);  
        for (j = k+1; j < i; j++)  
            DGEMM(RW,A[i][j],  
                R,A[i][k], R,A[j][k]);  
    }  
}
```



A runtime (StarPU) is responsible for task scheduling

- Respects dependencies, moves data around

StarVZ: visualization of task parallelism traces

Features

- *Post-mortem* analysis, after the parallel application finishes
- Has a series of visualization panels
- Fully based on the R/tidyverse analysis framework

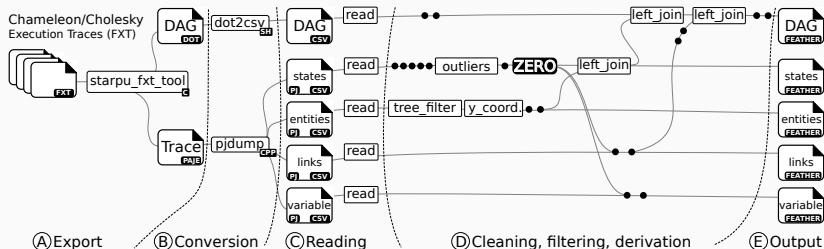


- Collection of packages → data science
- Share philosophy and grammar

A Visual Performance Analysis Framework for Task-based Parallel Applications running on Hybrid Clusters, Vinicius Garcia Pinto, Lucas Mello Schnorr, Luka Stanisic, Arnaud Legrand, Samuel Thibault, Vincent Danjean, in Concurrency and Computation: Practice and Experience, Wiley,

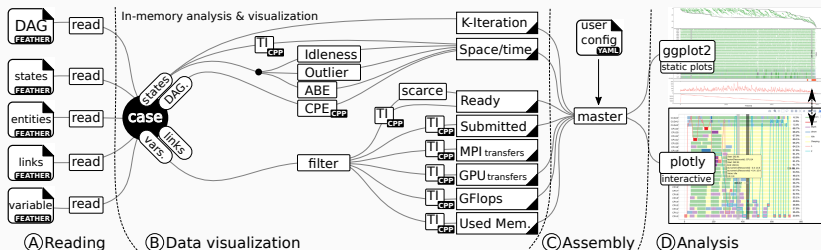
StarVZ: Phase 1

1. StarPU application is executed, generates `prof_file_X` files
 - Binary files (concern for low intrusiveness)
2. We run `starpu_fxt_tool`, a tool for exporting
3. We then run phase #1 of StarVZ
 - On the cluster → generate fast-read files



StarVZ: Phase 2

1. Data is typically brought to the Laptop
2. Fast reading of data in `parquet` format
3. Visualization panels, option to work directly with tables
 - Build your own analysis *script*

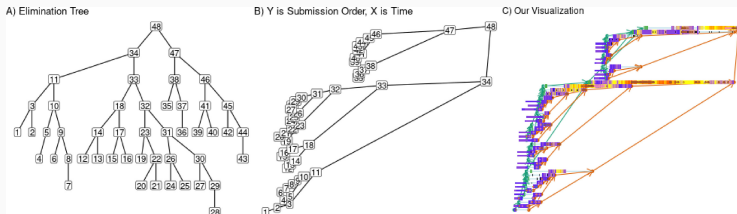


Usage examples

Sparse QR-Mumps Matrix Factorization 1/2

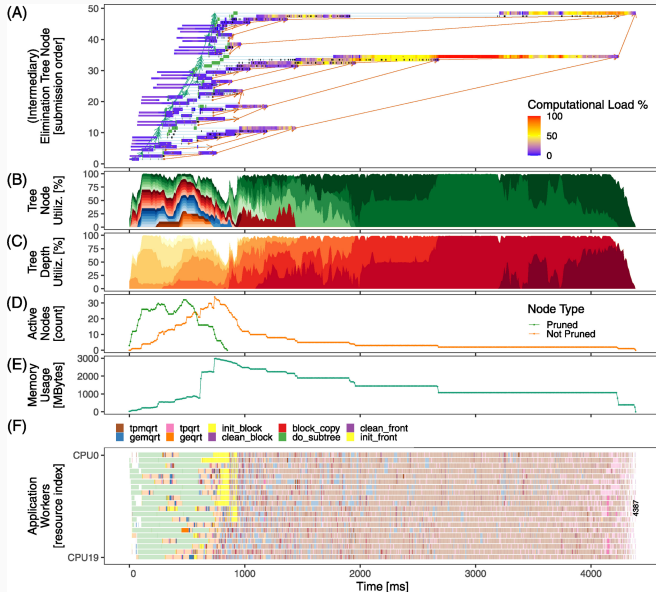
https://qr_mumps.gitlab.io/

Elimination tree to guide the factorization process



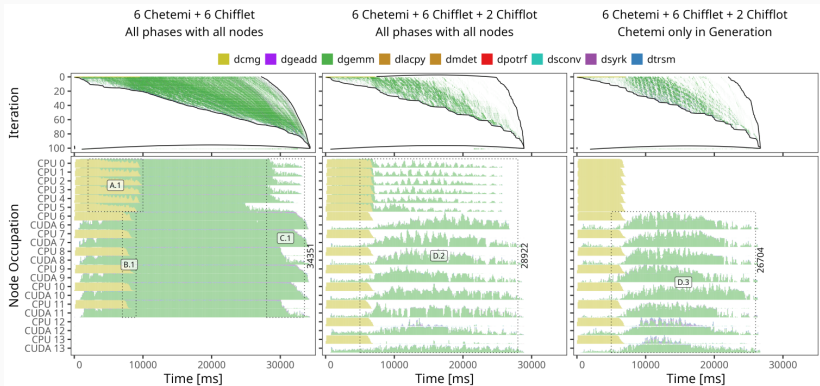
Miletto, Marcelo Cogo, et al. "Performance analysis of task-based multi-frontal sparse linear solvers: Structure matters." *Future Generation Computer Systems* 135 (2022): 409-425.

Sparse QR-Mumps Matrix Factorization 2/2



Cluster applications with ExaGeoStat on StarPU-MPI

<https://github.com/ecrc/exageostat>



Asynchronous multi-phase task-based applications: Employing different nodes to design better distributions. Future Gener. Comput. Syst. 147: 119-135 (2023)

Conclusion

Conclusions

Specific performance analysis per application

- Communication/computation overlap (MPI, CUDA)
- Identification of idle time reasons
- Identify scheduling delays
- Give analysts the freedom to test *what-if* scenarios

Conclusions

Specific performance analysis per application

- Communication/computation overlap (MPI, CUDA)
- Identification of idle time reasons
- Identify scheduling delays
- Give analysts the freedom to test *what-if* scenarios

Clear separation between tracing and analysis

- Using open formats and tools
- Allow the use of data science techniques
 - Efficient in-memory read formats
 - Use acceleration frameworks (RAPIDS, PySpark)

Conclusions

Specific performance analysis per application

- Communication/computation overlap (MPI, CUDA)
- Identification of idle time reasons
- Identify scheduling delays
- Give analysts the freedom to test *what-if* scenarios

Clear separation between tracing and analysis

- Using open formats and tools
- Allow the use of data science techniques
 - Efficient in-memory read formats
 - Use acceleration frameworks (RAPIDS, PySpark)

Favor the use of LLMs/SLMs

- Construction of analysis code snippets

Thank you for your attention!
schnorr@inf.ufrgs.br

Thanks to the VISTA EA Team

